

PHASE 1A: EVENT TICKETING PLATFORM

Sections 13, 14, 15 & 16 — Practices, Resources, Troubleshooting & Transition

TDD & Clean Code • All Dependencies & Resources • 7 Troubleshooting Guides • Phase 1B
Transition

This document covers the four closing sections of Phase 1A: the engineering practices that apply to every feature, the complete dependency and resource reference, a structured troubleshooting guide for the seven most common failure modes, and the full Phase 1A → 1B transition checklist. All code blocks have been converted to structured prose, visual cards, and reference tables.

SECTION 13

Practices Track Detail

TDD workflow, Clean Code checklist, Git branching strategy, commit conventions, and PR template

Practices are not optional polish — they are the habits that make complex codebases **maintainable under time pressure**. Every rule here has a specific reason. Each subsection explains not just what to do but **why it matters** for a project of this complexity.

13.1 TDD Workflow — Applied to Every Feature

Test-Driven Development (TDD) means writing the test **before** writing the implementation. This is not just about coverage — it forces you to design the method's contract (inputs, outputs, exceptions) before worrying about implementation details. The result is cleaner APIs and fewer edge-case bugs.

Why TDD for This Project Specifically

- **The state machine has 11 states and 9 events** — without tests written first, it is easy to wire a transition incorrectly and not notice until a user's booking gets stuck. TDD makes every transition a verifiable specification.
- **The PricingEngine has 4 compounding rules** — without a test for every combination, a pricing bug would only surface in production when a user is overcharged or undercharged.
- **The concurrency test is the only way to know the locking works** — you cannot reason about 100 simultaneous threads manually. The test is the specification.

The 6-Step Red-Green-Refactor Cycle

Step
1

Write the Test First

Before writing any production code, write a test that describes what the method **SHOULD** do. Use `@DisplayName` with a full English sentence. The test will not compile yet — that is correct.

Step
2

Run the Test — Watch It Fail (RED)

Run the test suite. It should fail — either a compilation error (method does not exist yet) or a runtime assertion failure. A test that passes before you write the implementation is a useless test.

Step
3

Write the Minimum Code to Pass

Write only enough production code to make the test pass. Resist the urge to add logic for other cases — that comes in later steps. If you write more than the minimum, you are not doing TDD.

Step
4

Run the Test — Watch It Pass (GREEN)

Run the test suite again. The test should now pass. If it does not, fix the implementation until it does. Do not move on until the test is green.

Step
5

Refactor — Clean Up Without Breaking

Now improve the code: rename variables, extract helper methods, remove duplication. Run the tests after every refactor to verify nothing broke. The tests are your safety net.

Step
6

Repeat for the Next Behaviour

Return to Step 1 for the next requirement. Each loop adds one testable behaviour. After 5–8 cycles, you have a fully tested, well-structured method.

TDD in Practice — `calculateRefundAmount()` Example

This walkthrough shows how TDD builds the `calculateRefundAmount()` method one behaviour at a time:

1. **Write the test:** Write a test named `calculateRefundAmount_fiveDaysBeforeEvent_shouldReturn50Percent()` annotated `@Test @DisplayName("calculateRefundAmount: should return 50% when cancelled 5 days before event")`. Arrange: build a `Booking` with `totalAmount = BigDecimal.valueOf(100.00)` and an `Event` whose `startDate = LocalDateTime.now().plusDays(5)`. Act: call `pricingEngine.calculateRefundAmount(booking, LocalDateTime.now())`. Assert: `assertThat(refund).isEqualByComparingTo("50.00")`.
2. **Run it — RED:** Compilation error — `calculateRefundAmount` does not exist yet. Create a stub that returns `BigDecimal.ZERO`. Now it compiles. Run again — assertion fails (`0.00 ≠ 50.00`). Still RED — correct.
3. **Implement — minimum:** Add the 3-to-7-day branch: if `daysUntilEvent >= 3 && daysUntilEvent < 7`, return `booking.getTotalAmount().multiply(BigDecimal.valueOf(0.50))`.
4. **Run it — GREEN:** Test passes. The 5-day case now returns 50.00.
5. **Repeat for other branches:** Write the ≥ 7 -day test (full refund), then the < 3 -day test (zero refund). Each follows the same RED → GREEN cycle.

13.2 Clean Code Checklist — Run Before Every Commit

These 14 rules apply to every commit pushed in Phase 1A. Most can be checked in under 5 minutes with a quick scan. **Do not merge code that violates these rules** — technical debt compounds quickly in a project of this complexity.

Rule	Pass Condition	Why It Matters
No method exceeds 20 lines	Extract methods	Methods over 20 lines are doing more than one thing. Extract named helper methods — each name documents intent.
No class exceeds 200 lines	Split class	A 300-line service class has multiple responsibilities. Split by extracting a new @Service class.
Every variable name describes what it holds	No a, b, temp, data	Code is read 10x more than written. Descriptive names make the reviewer's (and your future self's) job faster.
No magic numbers — use named constants (MAX_TICKETS_PER_BOOKING = 10)	No raw literals	When the business rule changes from 10 to 15, you find and fix one constant, not 8 scattered occurrences.
No raw Strings for status values — use enums (BookingState.RESERVED, not "RESERVED")	Enums only	Raw strings cause silent bugs: a typo like "RESEVED" compiles fine and causes a null comparison at runtime.
Single Responsibility: each class does ONE thing	One reason to change	If you need to change BookingService both when the pricing logic changes AND when the email format changes, it has two responsibilities.
No code duplication — if copied > 3 times, extract to method/utility	DRY principle	Duplicate logic means bugs get fixed in one copy and not the other.
All exceptions have meaningful messages — "Booking not found: id=" + id	Include context	Stack traces without context waste hours of debugging time. Include the ID, the state, the user — whatever makes the error self-explanatory.
@Transactional only on service layer — never on controllers or repositories	Service layer only	Repositories are already transactional. Controllers are not the place for transaction boundaries. Misplaced @Transactional silently has no effect (proxy bypass).
No @Autowired on fields — use constructor injection with @RequiredArgsConstructor	Constructor injection	Field injection cannot be used in tests without Spring context. Constructor injection makes dependencies explicit and testable with plain new.
All DTOs use @Builder (for readable test assertions)	@Builder present	Builder pattern makes test setup readable: Booking.builder().totalAmount(...).event(...).build() vs multiple setter calls.
No System.out.println — use @Slf4j: log.info(), log.debug(), log.error()	No sysout	System.out bypasses log level configuration and is invisible in production log aggregators. Structured logs are searchable.
Swagger @Operation annotation on every controller method	@Operation present	Swagger UI only shows useful documentation if every endpoint has a summary. This is part of the portfolio — reviewers will check it.
All non-obvious public methods have a brief Javadoc comment	Javadoc present	Future contributors (and your Phase 1B self) should not need to reverse-engineer what a method does from its internals.

13.3 Git Branching Strategy

All feature development happens on feature branches cut from `develop`. The `main` branch is protected — direct pushes are disabled. Every merge to `main` triggers the CI/CD pipeline and deploys to Railway.

Branch Hierarchy

Branch	Protected?	Purpose and Rules
<code>main</code>	YES — protected	Production branch. Only receives merges from <code>develop</code> via pull request after CI passes. Every push to <code>main</code> triggers Railway deployment.
<code>develop</code>	Soft protection	Integration branch. All feature branches merge here first. Run <code>./mvnw test</code> locally before opening a PR to <code>develop</code> .
<code>feature/event-crud</code>	No	Cut from <code>develop</code> . One branch per feature. Merged back to <code>develop</code> via PR when the feature is complete and all tests pass.
<code>feature/booking-state-machine</code>	No	State machine implementation branch.
<code>feature/stripe-integration</code>	No	Stripe checkout and webhook handler.
<code>feature/rabbitmq-notifications</code>	No	RabbitMQ publishers and listeners.
<code>feature/redis-distributed-locking</code>	No	DistributedLockService and concurrency tests.
<code>feature/pricing-engine</code>	No	PricingEngine and RefundService.
<code>feature/frontend-home-page</code>	No	Next.js home page and EventCard component.
<code>feature/frontend-event-detail</code>	No	Event detail, cart, and confirmation pages.
<code>feature/ci-cd-pipeline</code>	No	GitHub Actions workflows and Dockerfile.

13.4 Conventional Commit Format

Every commit must follow the **Conventional Commits** specification: `<type>(<optional scope>) : <description>`. This enables automatic changelog generation, makes `git log` readable, and signals to reviewers what kind of change a commit represents.

Type	Example Commit Message	When to Use
feat	<code>feat: implement booking state machine with guards and actions</code>	Adding new functionality — a new feature, endpoint, service, or component.
feat	<code>feat: add stripe checkout session creation</code>	Each distinct integration gets its own feat commit.
feat	<code>feat: add rabbitmq booking confirmation queue</code>	Infrastructure additions that introduce new capability count as features.
test	<code>test: add concurrent booking test with 100 threads</code>	Adding or updating tests only. No production code changes in this commit.
fix	<code>fix: release redis lock on payment failure</code>	Correcting a bug in existing code. Should reference what was broken.
fix	<code>fix: n+1 query in event list endpoint</code>	Performance bug fixes use fix, not perf (perf is for non-bug optimisations).
perf	<code>perf: add composite index on ticket_tiers(event_id, tier_name)</code>	Non-bug performance improvement — new index, connection pool tuning, caching.
refactor	<code>refactor: extract pricing rules to dedicated PricingEngine class</code>	Code restructuring with no behaviour change. Tests should still pass unchanged.
docs	<code>docs: add state machine diagrams to readme</code>	Documentation-only changes — README updates, Javadoc, architecture diagrams.
ci	<code>ci: add github actions workflow for test and build</code>	Changes to CI/CD configuration files — GitHub Actions, Dockerfile, scripts.
chore	<code>chore: upgrade spring boot to 3.2.4</code>	Maintenance tasks — dependency upgrades, build config, project setup.

13.5 Pull Request Description Template

Every PR to `develop` or `main` must include a description covering five sections. A well-written PR description is part of your portfolio — future employers may look at your GitHub PRs. It also makes code review faster because the reviewer understands the intent before reading a single line of code.

Section	Example Content
## What Changed	— Implemented BookingStateMachine with Spring State Machine — Added 8 state transitions: AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED — Added timer expiration job (runs every 30s, expires stale RESERVED bookings)
## Why	Part of the Phase 1A booking flow. This is the core business logic that prevents double-bookings and manages the ticket lifecycle.

Section	Example Content
## Key Decisions	— Used Spring State Machine over manual if/else for maintainability and auditability — Guard pattern validates seat availability before transition (cleaner than throwing inside the service) — State persisted to DB after each transition (stateless between requests — horizontally scalable)
## How to Test	1. Start the app: docker-compose up 2. POST /api/bookings/reserve (expect: 200, state=RESERVED) 3. Wait 5+ minutes without checkout 4. GET /api/bookings/{id} (expect: state=EXPIRED) OR: ./mvnw test -Dtest=BookingStateMachineTest
## Tests Added	— BookingStateMachineTest: 5 tests — ConcurrentBookingTest: 1 test (100 threads → exactly correct outcome) — BookingServiceIntegrationTest: 3 tests
## Coverage Delta	Before: 45% After: 58%

✓ **TIP:** Add a PR template file at .github/pull_request_template.md so the five sections appear automatically when opening a new PR on GitHub. This ensures every PR follows the same format without relying on memory.

SECTION 14

Tools & Resources

Complete Maven dependency reference, curated resource list by topic, and external tool links

This section is the single reference for every dependency and external resource needed during Phase 1A. The dependency table shows **all 20+ Maven dependencies** with their purpose and scope. The resource list is curated and prioritised — not a dump of search results.

14.1 Maven Dependencies — Complete pom.xml Reference

Spring Boot parent version: **3.2.4**. Set this in the `<parent>` block of `pom.xml`. All spring-boot-starter-* dependencies inherit their version from the parent — do not specify versions for these manually.

Spring Boot Core Starters

artifactId	groupId	Version	Scope	Purpose
spring-boot-starter-web	org.springframework.boot	(parent)	compile	REST API, embedded Tomcat, Jackson JSON serialization
spring-boot-starter-data-jpa	org.springframework.boot	(parent)	compile	Hibernate ORM, @Entity, @Repository, Spring Data repositories
spring-boot-starter-security	org.springframework.boot	(parent)	compile	Spring Security — JWT filter chain, @PreAuthorize, role-based access
spring-boot-starter-data-redis	org.springframework.boot	(parent)	compile	RedisTemplate, RedisCacheManager, @Cacheable, @CacheEvict
spring-boot-starter-amqp	org.springframework.boot	(parent)	compile	RabbitTemplate, @RabbitListener, RabbitMQ connection management
spring-boot-starter-validation	org.springframework.boot	(parent)	compile	Bean Validation — @NotBlank, @NotNull, @Future, @Min on DTOs
spring-boot-starter-actuator	org.springframework.boot	(parent)	compile	/actuator/health endpoint — used by Docker health checks and Railway
spring-boot-starter-mail	org.springframework.boot	(parent)	compile	JavaMailSender for sending HTML confirmation emails via Mailhog/SMTP

Spring State Machine

artifactId	groupId	Version	Scope	Purpose
spring-statemachine-core	org.springframework.statemachine	3.2.1	compile	@EnableStateMachineFactory, StateMachineConfigurerAdapter, guards, actions

Database

artifactId	groupId	Version	Scope	Purpose
postgresql	org.postgresql	(parent)	runtime	PostgreSQL JDBC driver — loaded at runtime by Hibernate
flyway-core	org.flywaydb	(parent)	compile	Database migration — runs V1_...sql through V9_...sql on startup

JWT Authentication

artifactId	groupId	Version	Scope	Purpose
jjwt-api	io.jsonwebtoken	0.11.5	compile	JWT token API — JwtBuilder, JwtParserBuilder interface
jjwt-impl	io.jsonwebtoken	0.11.5	runtime	JWT implementation — loaded at runtime via ServiceLoader
jjwt-jackson	io.jsonwebtoken	0.11.5	runtime	Jackson-based JWT serialization — required for claims handling

Stripe Payment Integration

artifactId	groupId	Version	Scope	Purpose
stripe-java	com.stripe	24.3.0	compile	Stripe SDK — Session.create(), Webhook.constructEvent(), Refund.create()

QR Code Generation

artifactId	groupId	Version	Scope	Purpose
core	com.google.zxing	3.5.2	compile	ZXing core — QRCodeWriter, BarcodeFormat, BitMatrix
javase	com.google.zxing	3.5.2	compile	ZXing Java SE — MatrixToImageWriter for Base64 PNG encoding

API Documentation

artifactId	groupId	Version	Scope	Purpose
springdoc-openapi-starter-webmvc-ui	org.springdoc	2.3.0	compile	Swagger UI at /swagger-ui.html — @Operation, @ApiResponse, @Schema

Utilities

artifactId	groupId	Version	Scope	Purpose
lombok	org.projectlombok	(parent)	optional	@Data, @Builder, @RequiredArgsConstructor, @Slf4j — compile-time annotation processing

Testing (scope: test only)

artifactId	groupId	Version	Scope	Purpose
spring-boot-starter-test	org.springframework.boot	(parent)	test	JUnit 5, Mockito, AssertJ, @SpringBootTest, @WebMvcTest — all-in-one test starter
spring-rabbit-test	org.springframework.amqp	(parent)	test	RabbitMQ test utilities — @RabbitListenerTest for listener verification
junit-jupiter	org.testcontainers	(parent)	test	@Testcontainers, @Container lifecycle management for JUnit 5
postgresql	org.testcontainers	(parent)	test	PostgreSQLContainer — real PostgreSQL in Docker during integration tests
rabbitmq	org.testcontainers	(parent)	test	RabbitMQContainer — real RabbitMQ in Docker during integration tests
awaitility	org.awaitility	(parent)	test	Awaitility.await().atMost(5, SECONDS) — async assertion without Thread.sleep()

14.2 Critical Resources — Organised by Topic

These resources are **curated**, not exhaustive. Each entry is here because it directly helps with a specific Phase 1A implementation task. Read or watch each one **at the point in the schedule when you are building that feature** — not all at once at the start.

Spring State Machine

URL / Search Term	Format	What to Focus On
https://docs.spring.io/spring-statemachine/docs/current/reference/	Official reference docs	Read before Day 8. Focus on StateMachineConfigurerAdapter, guards, actions, and @EnableStateMachineFactory vs @EnableStateMachine.
https://www.baeldung.com/spring-state-machine	Baeldung tutorial	Practical walkthrough. Read the "Events and Transitions" and "Actions and Guards" sections specifically.
https://github.com/spring-attic/spring-statemachine-samples	Spring SM samples on GitHub	Working code examples for complex state machines. Use as reference if the config structure is unclear.
YouTube — search "Spring State Machine Tutorial" by Amigoscode	YouTube (~45 min)	Video walkthrough. Watch after reading the docs for a different perspective on the same concepts.

Stripe Integration (Java)

URL / Search Term	Format	What to Focus On
https://stripe.com/docs/checkout/quickstart?lang=java	Checkout quickstart (Java)	The step-by-step guide for creating Checkout Sessions in Java. Follow this exactly on Day 9.
https://stripe.com/docs/webhooks	Webhooks guide	Signature verification, event types, idempotency. Read before implementing StripeWebhookController.
https://stripe.com/docs/testing	Test mode reference	All test card numbers, webhook trigger commands, and test clock tools. Keep this open during Day 9–10.
https://stripe.com/docs/stripe-cli	Stripe CLI docs	Install before Day 9. The listen --forward-to command is required for local webhook testing.

RabbitMQ

URL / Search Term	Format	What to Focus On
https://docs.spring.io/spring-amqp/docs/current/reference/html/	Spring AMQP reference	Complete reference for @RabbitListener, RabbitTemplate, exchange/queue declarations, and manual ACK.

URL / Search Term	Format	What to Focus On
https://www.rabbitmq.com/tutorials/tutorial-one-java	RabbitMQ Java tutorials	Official step-by-step tutorials covering topic exchanges, dead letters, and message persistence.
https://www.rabbitmq.com/dlx.html	Dead Letter Exchanges (DLX)	Read before configuring queues on Day 5. Explains the x-dead-letter-exchange argument and failure routing.
YouTube — search "Spring Boot RabbitMQ Tutorial" by TechPrimers	YouTube (~30 min)	Good visual explanation of exchanges, queues, and bindings before implementing them.

Redis

URL / Search Term	Format	What to Focus On
https://redis.io/docs/manual/patterns/distributed-locks/	Distributed locks pattern	The authoritative explanation of SET NX EX vs SETNX + EXPIRE, and the Lua release script. Read before Day 11.
https://redis.io/docs/manual/patterns/distributed-locks/#the-redlock-algorithm	Redlock algorithm	Read the trade-offs of single-node vs multi-node locks. Document your decision in the README.
https://docs.spring.io/spring-data/redis/reference/redis.html	Spring Data Redis reference	RedisTemplate configuration, @Cacheable, RedisCacheManager. Read before Day 5.
https://redis.io/commands/	Redis command reference	Bookmark this. Use it to look up exact syntax for SET, ZADD, ZPOPMIN, HSET, INCR, DECR as needed.
YouTube — search "Redis Distributed Lock Explained" by ByteByteGo	YouTube (~12 min)	Visual walkthrough of the lock/release pattern. Excellent 12-minute primer before Day 11.

Testing

URL / Search Term	Format	What to Focus On
https://testcontainers.com/guides/testing-spring-boot-rest-api-using-testcontainers/	Testcontainers Spring Boot guide	The canonical getting-started guide. Follow this to set up BaseIntegrationTest on Day 6.
https://www.jacoco.org/jacoco/trunk/doc/maven.html	JaCoCo Maven plugin	Configuration reference for the prepare-agent, report, and check executions. Use on Day 16.
https://k6.io/docs/	k6 documentation	Complete reference for writing load test scripts, configuring thresholds, and interpreting output.
https://k6.io/docs/get-started/installation/	k6 installation	Install before Day 17. Available via Homebrew (macOS) or binary download for Linux/Windows.

System Design

URL / Search Term	Format	What to Focus On
Designing Data-Intensive Applications (DDIA) — Chapters 1–3	Book	The best software book for understanding distributed systems. Read Ch. 1 on Day 1, Ch. 2 before Week 2, Ch. 3 before Week 3.
https://systemdesign.one/ticketmaster-system-design/	System Design One — Ticketmaster	Reference solution for the Design Ticketmaster exercise (Day 14). Compare with your own diagram after completing it.
https://www.educative.io/courses/grokking-the-system-design-interview	Grokking System Design (Educative)	Structured course with design problems and solutions. Use for additional practice after Ticketmaster and Eventbrite.
YouTube — search "Stripe Checkout Java" on Stripe official channel	YouTube (~20 min)	Stripe's own tutorial video for Java checkout integration. Watch alongside Day 9 work.

14.3 External Accounts and Tooling Setup

These accounts and tools must be set up before their respective implementation days:

Tool/Account	Setup URL	Needed By	What You Need From It
GitHub	github.com	Day 1	Public repository for the project. Enable branch protection on main.
Stripe (test mode)	stripe.com	Day 9	sk_test ... secret key and webhook signing secret (from Stripe CLI)
Stripe CLI	stripe.com/docs/stripe-cli	Day 9	Local webhook forwarding: stripe listen --forward-to localhost:8080/...
Railway	railway.app	Day 18	Free hosting with PostgreSQL, Redis, RabbitMQ plugins. RAILWAY_TOKEN for CI/CD.
Vercel (frontend)	vercel.com	Day 18	Free Next.js hosting. Links to the Railway backend via NEXT_PUBLIC_API_URL.
Loom	loom.com	Day 20	3-minute demo video recording for README and LinkedIn portfolio.
Excalidraw	excalidraw.com	Day 1	Free online diagramming for the DB ERD and system architecture diagrams.

SECTION 15

Troubleshooting Guide

The 7 most common failure modes — each with symptom, root cause, diagnosis steps, and fix

Every issue listed here was identified as a likely failure point during Phase 1A development. The structure for each issue is: **Symptom** (what you observe), **Root Cause** (why it happens), **Fix** (exact steps to resolve). Severity levels: **HIGH** = blocks all progress, **MEDIUM** = blocks a feature, **LOW** = irritant but workable.

#1 — Stripe Webhook Not Receiving Events [HIGH]

Symptom	Payment completes successfully on Stripe's hosted page but the booking remains in PAYMENT_PENDING state. The confirmation page never loads.
Root Cause	Stripe cannot reach localhost:8080 from the internet. Webhooks require a publicly accessible URL or a local forwarding tool. Without forwarding, Stripe delivers the event to a URL that points nowhere.
Fix	Option A — Stripe CLI (recommended): Install with <code>brew install stripe/stripe-cli/stripe</code>. Login with <code>stripe login</code>. Run <code>stripe listen --forward-to localhost:8080/api/webhooks/stripe</code>. Copy the <code>whsec_...</code> signing secret printed to the console and set it as <code>STRIPE_WEBHOOK_SECRET</code> in <code>application-local.yml</code>. Option B — ngrok: Run <code>ngrok http 8080</code>. Copy the <code>https://xxxxx.ngrok.io</code> URL. In Stripe Dashboard → Developers → Webhooks → Add endpoint, paste the URL with <code>/api/webhooks/stripe</code> appended. Subscribe to events <code>checkout.session.completed</code> and <code>checkout.session.expired</code>. Verify: Stripe CLI prints each delivered event. After completing a test payment, you should see <code>checkout.session.completed</code> appear in the CLI output within 2 seconds.
Note	<i>The Stripe CLI is preferred because it handles the signing secret automatically and does not require a Stripe Dashboard webhook endpoint configuration. Restart the CLI whenever <code>STRIPE_WEBHOOK_SECRET</code> is not set correctly.</i>

#2 — RabbitMQ Connection Refused on App Startup [HIGH]

Symptom	Spring Boot fails to start with <code>AmqpConnectException: Connection refused to localhost:5672</code> . All RabbitMQ-dependent features are broken.
Root Cause	The RabbitMQ container is not running, is still starting, or did not pass its health check before the app tried to connect.
Fix	Verify RabbitMQ container status: run <code>docker ps grep rabbit</code>. If the container is absent, run <code>docker-compose up -d rabbitmq</code>.

	10. Check container health: run <code>docker ps</code> and look at the STATUS column. It should say <code>(healthy)</code>. If it shows <code>(starting)</code>, wait 30 seconds and check again. 11. Check container logs if unhealthy: <code>docker logs ticketing_rabbitmq_1</code>. Look for errors about disk space, port conflicts, or memory limits. 12. Verify queue declarations: once healthy, open the RabbitMQ Management UI at <code>http://localhost:15672</code> (user: ticketing, pass: ticketing_pass). Click Queues tab — all 3 queues and 3 DLQs should appear automatically on app startup. 13. If queues are missing: the <code>RabbitMQConfig.java</code> beans may not be loading. Check Spring logs for <code>AmqpAdmin</code> errors or missing <code>@Configuration</code> annotation.
Note	Add condition: <code>service_healthy</code> to the app's <code>depends_on</code> block in <code>docker-compose.yml</code> . This prevents the app from starting until RabbitMQ passes its health check.

#3 — Redis Lock Deadlock — Lock Never Released [HIGH]

Symptom	After a failed booking attempt, all subsequent reservations for the same tier immediately fail with <code>LockAcquisitionException</code> . The tier appears permanently locked.
Root Cause	The <code>releaseLock()</code> method was not called because the code crashed before reaching the finally block, or the lock was acquired without a TTL (missing <code>NX EX</code> — lock persists forever).
Fix	14. Inspect live lock keys: run <code>redis-cli keys "seat:lock:*"</code>. If lock keys are present, check their TTL with <code>redis-cli ttl "seat:lock:tier:1:user:123"</code>. A TTL of <code>-1</code> means the key was set without an expiry — this confirms the root cause. 15. Manually remove a stuck lock (dev only): <code>redis-cli del "seat:lock:tier:1:user:123"</code>. 16. Fix the root cause — verify <code>executeWithLock()</code> has a finally block: the structure must be <code>try { return operation.get(); } finally { releaseLock(key, value); }</code>. The finally block runs even if the operation throws an exception. 17. Verify atomic acquisition: confirm the <code>acquireLock()</code> method uses <code>redisTemplate.opsForValue().setIfAbsent(key, value, Duration.ofSeconds(ttl))</code> — the single atomic <code>SET NX EX</code> command. If it uses separate <code>SETNX</code> and <code>EXPIRE</code> calls, replace them.
Note	In <code>redis-commander</code> (<code>http://localhost:8081</code>), filter keys by <code>seat:lock:</code> prefix to monitor locks in real time during development. All lock keys should disappear within 300 seconds even if never explicitly released, because the TTL acts as a dead-man's switch.

#4 — N+1 Query Still Present After Attempted Fix [MEDIUM]

Symptom	Loading a list of 10 events generates 31 or more SQL statements in the logs, even after adding <code>@EntityGraph</code> or <code>JOIN FETCH</code> .
Root Cause	The fix was applied to the wrong repository method, or the controller/service is calling a different method than the one with the fix. Spring JPA proxies mean a method called on the same class (self-invocation) bypasses <code>@Transactional</code> and <code>@EntityGraph</code> .
Fix	18. Enable SQL logging first: add <code>spring.jpa.show-sql=true</code> and <code>spring.jpa.properties.hibernate.format_sql=true</code> to <code>application-local.yml</code>. Restart the app and count the SQL statements for a list fetch. The target is ≤ 3 queries for 10 events. 19. Fix 1 — <code>@EntityGraph</code> on the repository method called by the service: <code>@EntityGraph(attributePaths = {"organizer", "category", "venue"}) List<Event> findByStatus(EventStatus status).</code> Confirm it is the exact method signature the service calls. 20. Fix 2 — JPQL with <code>JOIN FETCH</code>: <code>@Query("SELECT e FROM Event e JOIN FETCH e.organizer JOIN FETCH e.category JOIN FETCH e.venue WHERE e.status = :status") List<Event> findByStatusWithDetails(@Param("status") EventStatus status).</code> 21. Fix 3 — for <code>@OneToMany</code> collections (tickets in booking): enable batch fetching with <code>spring.jpa.properties.hibernate.default_batch_fetch_size=10.</code> This replaces $N+1$ with $(N/10)+1$ queries — much better for collections. 22. Run <code>EXPLAIN ANALYZE</code> in PostgreSQL on the query: connect to the DB (psql or pgAdmin), run <code>EXPLAIN ANALYZE <the query></code>, and verify it shows Index Scan not Seq Scan.
Note	<i>Document the before and after query counts in the README Performance section. This is concrete evidence for interviewers that you understand the $N+1$ problem and know how to fix it.</i>

#5 — @Transactional Not Rolling Back on External Service Failure [MEDIUM]

Symptom	When a Stripe API call fails, the seat inventory in the database has already been decremented and is not rolled back. Tickets are permanently lost.
Root Cause	Spring <code>@Transactional</code> only rolls back changes to the Spring-managed <code>DataSource</code> . Redis operations, Stripe API calls, and RabbitMQ publishes are external — they are not part of the DB transaction and do not roll back automatically.
Fix	23. Understand the proxy bypass problem: <code>@Transactional</code> uses Spring AOP proxies. If <code>reserveTickets()</code> calls <code>reduceInventory()</code> on the SAME class, it goes through the same object (not the proxy) — <code>@Transactional</code> on the inner method is silently ignored. 24. Fix — move the inner method to a separate <code>@Service</code> class: <code>BookingService</code> injects <code>InventoryService</code> as a dependency. When <code>bookingService.reserveTickets()</code> calls <code>inventoryService.releaseSeat()</code>, it goes through the Spring proxy — <code>@Transactional</code> applies.

	25. For external services (Stripe, Redis): use compensating transactions. Wrap the Stripe call in a try-catch. In the catch block, manually reverse the DB change: call <code>inventoryService.releaseSeat()</code> before rethrowing the exception. Log the compensating action. 26. Add Spring Retry for optimistic locking failures: annotate <code>reserveTickets()</code> with <code>@Retryable(value = OptimisticLockingFailureException.class, maxAttempts = 3, backoff = @Backoff(delay = 100, multiplier = 2))</code>. Add <code>@EnableRetry</code> to the main application class and add <code>spring-retry</code> to <code>pom.xml</code>.
Note	<i>The <code>ObjectOptimisticLockingFailureException</code> from the <code>@Version</code> field is EXPECTED — it means the optimistic locking is working correctly. Handle it with <code>@Retryable</code> rather than removing the <code>@Version</code> field.</i>

#6 — Docker Compose Services Start Out of Order [MEDIUM]

Symptom	Spring Boot starts before PostgreSQL, Redis, or RabbitMQ is ready. The app throws Connection refused on startup and exits.
Root Cause	Docker Compose <code>depends_on</code> without condition only waits for the container to START, not for it to be ready. PostgreSQL takes ~3 seconds to accept connections after starting. Spring Boot attempts connection within ~1 second of the container starting.
Fix	27. Add condition: <code>service_healthy</code> to the app's <code>depends_on</code> in <code>docker-compose.yml</code>: 28. <code>depends_on</code>: with sub-keys <code>postgres</code>: <code>condition</code>: <code>service_healthy</code>, <code>redis</code>: <code>condition</code>: <code>service_healthy</code>, <code>rabbitmq</code>: <code>condition</code>: <code>service_healthy</code>. 29. Verify each service has a healthcheck defined: PostgreSQL should use <code>pg_isready -U ticketing</code>, Redis should use <code>redis-cli ping</code>, RabbitMQ should use <code>rabbitmq-diagnostics ping</code>. 30. Add HikariCP retry to <code>application.yml</code> as a second safety net: <code>spring.datasource.hikari.connection-timeout: 30000</code> and <code>spring.datasource.hikari.initialization-fail-timeout: 60000</code>. This gives HikariCP 60 seconds to acquire the first connection before failing.
Note	<i>Run <code>docker-compose up --build</code> and observe the logs. The app container should print "Waiting for postgres to be healthy..." before the Spring Boot startup banner appears. If it starts immediately, the <code>depends_on</code> condition is not configured correctly.</i>

#7 — ObjectOptimisticLockingFailureException Under Concurrent Load [LOW]

Symptom	Concurrent POST <code>/api/bookings/reserve</code> requests intermittently throw <code>ObjectOptimisticLockingFailureException</code> . Some booking attempts fail unexpectedly.
----------------	--

Root Cause	This is the optimistic locking working as designed — two threads read the same @Version value and one of them wins the DB update while the other gets rejected. This is a correct safety mechanism, not a bug. The application needs retry logic to handle the rejected thread gracefully.
Fix	31. Add the spring-retry dependency to pom.xml: <code>spring-retry:spring-retry</code> with no explicit version (inherited from Spring Boot parent). 32. Add <code>@EnableRetry</code> to <code>TicketingApplication.java</code>. 33. Annotate <code>reserveTickets()</code> with <code>@Retryable(value = {OptimisticLockingFailureException.class}, maxAttempts = 3, backoff = @Backoff(delay = 100, multiplier = 2.0))</code>. This retries the method up to 3 times with 100ms, 200ms, 400ms delays between attempts. 34. Verify the concurrent booking test still passes: the <code>@Retryable</code> annotation should not change the outcome of <code>ConcurrentBookingTest</code> — exactly 50 threads should succeed and 50 should fail (after exhausting retries).
Note	<i>ObjectOptimisticLockingFailureException appearing in logs during the concurrent test is NORMAL and expected. The @Version field on Booking is the second line of defence after Redis locks. Do not remove it — it is the safety net when Redis fails.</i>

SECTION 16

Phase 1A → Phase 1B Transition

Completion checklist, portfolio artifacts, self-assessment, code cleanup, documentation, and Phase 1B direction

Phase 1A is complete when every item in this section is checked off. The transition to Phase 1B should not happen until the codebase is clean, the deployment is live, and the portfolio artifacts are published. **A rushed Phase 1A closeout creates technical debt that makes Phase 1B harder.**

16.1 Technical Deliverables — Required Before Phase 1B

GitHub Repository

- ❑ **Repository is public** with the full commit history visible (not a single squashed commit).
- ❑ **README.md is complete** with all sections from Section 19 plan: architecture diagram, state machine diagrams, screenshots, quick-start guide, env vars table, load test results, What I Learned.
- ❑ **At least 5 UI screenshots** are embedded in the README — home page, event detail, cart with countdown, confirmation with QR code, and organizer dashboard.
- ❑ **Architecture diagram** (Excalidraw or draw.io) embedded in README — CDN, LB, API servers, DB, Redis, RabbitMQ.
- ❑ **State machine diagram** (visual, not just code) embedded in README — all 11 states and 9 transitions.

Live Deployment

- ❑ `https://{your-app}.up.railway.app/actuator/health` returns `{"status": "UP"}`.
- ❑ Swagger UI accessible at `https://{your-app}.up.railway.app/swagger-ui.html` with all endpoints documented.
- ❑ Live deployment stays up without sleeping (Railway free tier stays active unless there is zero traffic for a long period — ping it periodically).
- ❑ End-to-end booking flow works on live URL: browse → select → cart → Stripe test payment → confirmation with QR code.

Quality Gates

- ❑ `./mvnw clean verify` passes with JaCoCo coverage $\geq 80\%$ (build fails otherwise).
- ❑ All 13 test classes pass — `./mvnw clean test` with zero failures and zero test errors.
- ❑ k6 load test results documented in README: P95 < 500ms, zero double-bookings, error rate < 1%.

- All 33 success criteria checkboxes from Section 2's weekly checkpoints are verified.

16.2 Portfolio Artifacts

Required Before Phase 1B

- **3-minute Loom demo video** showing the full flow: browse events → select tier → cart countdown → Stripe payment → confirmation with QR code → organizer dashboard showing revenue. Share the URL in the README.
- **LinkedIn project entry** created: title "Event Ticketing Platform", description (2–3 sentences), tech stack, links to GitHub repo and live demo URL.
- **"What I Built" paragraph** written and ready for resume, cover letters, and interviews. Approximately 3 sentences covering the system, the hardest technical challenge, and the measurable outcome.

Resume Bullet Point Examples

Use these as starting points — personalise with your actual numbers and decisions:

- "Designed and implemented a high-concurrency event ticketing platform handling 1,000 concurrent users with Redis distributed locking and Spring State Machine, achieving P95 response time < 500ms with zero oversell incidents."
- "Built end-to-end Stripe payment integration with webhook idempotency, partial refund processing, and a 5-tier booking state machine enforcing legal state transitions across 11 states."
- "Achieved 80%+ JaCoCo test coverage across a Java Spring Boot backend with 13 test classes, including a 100-thread concurrent booking test verifying zero race conditions in Redis-backed inventory management."

16.3 Self-Assessment — Rate Yourself 1–5 Per Skill

Rate yourself honestly on each skill before starting Phase 1B. This is not for external review — it shapes which Phase 1B project to choose and where to focus early in Phase 1B. A **1** means "I need to look this up every time". A **5** means "I can explain this clearly in an interview and implement it from scratch".

Transaction Management	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

State Machines	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

Payment Integration	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

Message Queues	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

Redis Patterns	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

Concurrency & Performance	1	2	3	4	5
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐
	☐	☐	☐	☐	☐

16.4 Code Cleanup Checklist — Run Before Closing Phase 1A

Run these commands in sequence. All must pass before Phase 1B begins:

- `./mvnw clean verify` — runs all tests AND enforces JaCoCo $\geq 80\%$ coverage. The build must succeed.
- `./mvnw spring-boot:run` — app must start cleanly with no WARN or ERROR lines in the startup log (info-level startup is fine).

- `docker-compose up --build` — full Docker build must succeed and all services must reach (healthy) status within 2 minutes.
- `grep -r "TODO\|FIXME\|HACK\|System.out" src/main/java` — must return nothing. All TODOs must be resolved or removed before Phase 1B.
- `grep -r "sk_test_\|sk_live_" src/` — must return nothing. No Stripe secrets in source code. All secrets go through environment variables.

NOTE: Tag the Phase 1A release: `git tag -a v1.0.0 -m "Phase 1A complete: Event Ticketing Platform"` then `git push --tags`. This creates a permanent reference point on GitHub and makes the portfolio entry look professional.

16.5 Documentation to Finalise

These documentation files must exist in the repository by April 24:

File	Required Contents
<code>README.md</code>	Project title and description, live demo URL, tech stack badges, architecture diagram, state machine diagrams, quick start (<code>git clone</code> → <code>docker-compose up</code> → <code>localhost:3000</code>), env vars table, running tests, load test results and screenshot, project structure tree, What I Learned section, UI screenshots (5+), Loom video link
<code>docs/architecture.md</code>	System architecture description — CDN, API, databases, Redis, RabbitMQ. Explain why each component exists and what it does. Link to the architecture diagram image.
<code>docs/state-machines.md</code>	Visual diagrams for both the Booking state machine (11 states, 9 events) and the Event status lifecycle. Explain each transition in one sentence. This is frequently asked in interviews.
<code>docs/api.md</code>	Link to the live Swagger UI at <code>/swagger-ui.html</code> . List of all API endpoints with method, path, auth requirement, and brief description.
<code>docs/deployment.md</code>	Step-by-step instructions for setting up the project from scratch: prerequisites, local dev (<code>docker-compose up</code>), and Railway production deployment. Written for someone who has never seen the project.
<code>docs/load-testing.md</code>	k6 test script summary, command to run the test, screenshot of k6 output, P50/P95/P99 response times, throughput (req/sec), error rate, and the SQL verification query confirming zero oversell.

16.6 Phase 1B Direction — Decide by April 25

Three options are available for Phase 1B. Choose based on what you want to **learn next**, not just what you feel comfortable with. The goal is to expand the skill set.

Option	Project	Recommendation	New Skills Introduced
A	Social Media Analytics Platform	RECOMMENDED — builds directly on Phase 1A async patterns	Spring Batch, data pipelines, graph relationships (followers/following), event sourcing, deeper Redis patterns, analytics aggregation
C	E-Commerce Platform	RECOMMENDED — reinforces and extends payment patterns from Phase 1A	Inventory management (deeper), product search with Elasticsearch, recommendation engine, order management, vendor onboarding flow
B	Real-Time Collaborative Whiteboard	DEFER to Phase 3+ — reactive and WebSocket concepts need more foundation first	Spring WebFlux (reactive), WebSockets, CRDT concepts, operational transformation, more advanced frontend state management

 **IMPORTANT:** Before choosing: write down honestly what Phase 1A left unfinished and what was hardest. If distributed concurrency and async messaging felt shaky, Option A deepens exactly those patterns. If payment flows and inventory felt weak, Option C reinforces them.

SECTION APPENDIX

Daily Hours Log Template

Track actual vs planned hours daily — this data shapes Phase 1B planning and reveals patterns in your productivity

Track hours **honestly** every day. The target totals are: Project 62–68 hrs | Fundamentals (System Design) 22–27 hrs | Practices (TDD, docs, Git) 14–17 hrs | Total 105–122 hrs. If actual hours consistently fall short of plan, that tells you something important about the schedule assumptions for Phase 1B.

NOTE: Update this log at end of each day before closing your laptop. Estimated hours recorded the next morning are inaccurate — memory compresses unproductive time.

Date	Day Focus	Est Hrs	Project Hrs	Fundmtls Hrs	Practices Hrs
WEEK 1 — April 4–10 (Target: 35–40 hrs)					
Apr 4 (Sat)	Project init + DB schema	8			
Apr 5 (Sun)	Auth + Event Service (TDD)	7			
Apr 6 (Mon)	Category + Venue + Search	5			
Apr 7 (Tue)	Next.js init + Home page	5			
Apr 8 (Wed)	Inventory + Redis cache + RabbitMQ config	5			
Apr 9 (Thu)	Integration tests + N+1 fixes	5			
Apr 10 (Fri)	Week 1 cleanup + code review	5			
WEEK TOTAL		Target	Project	Fundmtls	Practices
WEEK 2 — April 11–17 (Target: 35–40 hrs)					
Apr 11 (Sat)	Booking state machine (core)	8			
Apr 12 (Sun)	Stripe integration	7			
Apr 13 (Mon)	RabbitMQ queues + email notifications	5			
Apr 14 (Tue)	Redis distributed locking + concurrent tests	5			
Apr 15 (Wed)	Pricing engine + refund logic + waitlist	5			
Apr 16 (Thu)	Frontend — Event Detail + Cart + Checkout	5			
Apr 17 (Fri)	Week 2 integration tests + system design	5			

Date	Day Focus	Est Hrs	Project Hrs	Fundmtls Hrs	Practices Hrs
WEEK TOTAL		Target	Project	Fundmtls	Practices
WEEK 3 — April 18–24 (Target: 28–32 hrs)					
Apr 18 (Sat)	Remaining frontend pages	7			
Apr 19 (Sun)	Test coverage push to 80%+	7			
Apr 20 (Mon)	k6 load testing + performance tuning	5			
Apr 21 (Tue)	CI/CD + Docker optimisation	5			
Apr 22 (Wed)	Documentation + Swagger + README	5			
Apr 23 (Thu)	Bug bash + demo preparation	5			
Apr 24 (Fri)	Final checkpoint + Phase 1B planning	3			
WEEK TOTAL		Target	Project	Fundmtls	Practices
PHASE 1A TOTAL		105–122 hrs	62–68 hrs	22–27 hrs	14–17 hrs